

Documentation du Système LangChain

Dual Book Advisor - Agent Intelligent de Recommandation

Version: 1.0

Date: 15/07/2025

Architecture: LangChain + LangGraph + RAG

Modèle: Llama 3.2 3B (Ollama)

Framework: Django + ChromaDB

Table des Matières

1. Fondamentaux de LangChain	3
1.1 Concepts de base	3
1.2 Nodes et Chains	3
1.3 LangGraph et orchestration	3
2. Architecture du Système	8
2.1 Vue d'ensemble	8
2.2 Structure modulaire	8
2.3 Composants principaux	8
3. Implémentation RAG	15
3.1 Système multi-domaines	15
3.2 ChromaDB et embeddings	15
3.3 Outils de recherche	15
4. Analyse des Workflows	22
4.1 Classification intelligente	22
4.2 Routage adaptatif	22
4.3 Formatage des réponses	22
5. Détails Techniques	28
5.1 Configuration	28
5.2 Performance	28
5.3 Monitoring	28
6. Guide de Migration	33
6.1 Stratégie progressive	33
6.2 Compatibilité	33
6.3 Tests et validation	33
7. Annexes	38
7.1 Code exemples	38
7.2 Configuration complète	38
7.3 Troubleshooting	38

1. Fondamentaux de LangChain

1.1 Concepts de Base

LangChain est un framework révolutionnaire pour le développement d'applications basées sur des modèles de langage (LLM). Il fournit une abstraction puissante pour orchestrer des workflows complexes impliquant des LLM, des bases de données vectorielles, et des outils externes.

Concept	Description	Usage dans notre système
LLM	Large Language Model - Le cœur du système	Llama 3.2 3B via Ollama
Chain	Séquence d'opérations liées	Classification → Routing → Formatting
Tool	Fonctions externes appelables	RAG Search Tools (Tech, Literature, Manga)
Prompt	Template de requête vers le LLM	Prompts spécialisés par domaine
Memory	Gestion de l'historique	State persistant via TypedDict
Agent	Entité autonome de décision	TechAgent, LiteratureAgent, MangaAgent

1.2 Nodes et Chains

Nodes (Nœuds):

Les nodes sont les unités de traitement atomiques dans LangChain. Chaque node effectue une tâche spécifique et peut être composé avec d'autres nodes pour former des workflows complexes.

Types de Nodes dans notre système:

- **ClassifieurNode:** Analyse et catégorise les requêtes utilisateur
- **RouterNode:** Dirige vers l'agent spécialisé approprié
- **AgentNodes:** Agents spécialisés (Tech, Literature, Manga)
- **FormatterNode:** Formate la réponse finale pour l'utilisateur

Chains (Chaînes):

Les chains connectent plusieurs nodes pour créer un workflow. Notre système utilise LangGraph pour créer des chaînes sophistiquées avec branchement conditionnel.

Exemple d'implémentation:

```
# Exemple de définition d'un Node
class ClassifieurNode:
    def __init__(self, llm):
        self.llm = llm
        self.parser = PydanticOutputParser(
            pydantic_object=QueryClassification
        )

    def run(self, state: BookAdvisorState) -> BookAdvisorState:
        # Classification de la requête
        classification = self._classify_query(state["user_query"])
        state["classification"] = classification
        return state
```

1.3 LangGraph et Orchestration

LangGraph est une extension de LangChain qui permet de créer des workflows sous forme de graphes dirigés. Il offre plusieurs avantages par rapport aux chains linéaires:

Avantages de LangGraph:

- **Branchement conditionnel:** Routage dynamique basé sur le contexte
- **Parallélisation:** Exécution simultanée de branches indépendantes
- **État persistant:** Gestion sophistiquée de l'état entre les nodes
- **Cycle de vie:** Contrôle fin du workflow avec points d'arrêt
- **Debugging:** Introspection et monitoring avancés

Notre Graph Workflow:

1. **START** → Initialisation de l'état
2. **Classifier** → Analyse de la requête (règles + LLM)
3. **Router** → Sélection de l'agent approprié
4. **Agent** → Traitement spécialisé (Tech/Literature/Manga)
5. **Formatter** → Mise en forme de la réponse
6. **END** → Retour de la réponse finale

2. Architecture du Système

2.1 Vue d'Ensemble

Le système Dual Book Advisor implémente une architecture sophistiquée combinant:

Stack Technologique:

- **Framework:** Django (backend web)
- **Orchestration:** LangChain + LangGraph
- **LLM:** Llama 3.2 3B (via Ollama)
- **Vector DB:** ChromaDB (embeddings et similarité)
- **RAG:** Système multi-domaines (Tech, Literature, Manga)

Principes de Design:

- **Modularité:** Séparation claire des responsabilités
- **Extensibilité:** Architecture pluggable pour nouveaux domaines
- **Robustesse:** Gestion d'erreurs et fallbacks
- **Performance:** Optimisations pour modèles locaux
- **Migration:** Compatibilité avec l'ancien système

2.2 Structure Modulaire

Hierarchie des Modules:

```
agents/langchain_agents/ ■■■ _init_.py # Module principal ■■■
django_integration.py # Bridge Django ↔ LangChain ■■■ graph_manager.py #
Orchestrateur LangGraph ■■■ nodes/ # Nœuds de traitement ■■■
agent_nodes.py # Agents spécialisés ■■■ classifieur_node.py #
Classification intelligente ■■■ tools/ # Outils externes ■■■ rag_tools.py
# Intégration RAG ■■■ chains/ # Chaînes (réservé) rags/ # Systèmes RAG
spécialisés ■■■ tech_rag/ # RAG technique ■■■ literature_rag/ # RAG
littérature ■■■ manga_rag/ # RAG manga/BD apps/chat/ # Interface Django ■■■
views_langchain.py # Endpoints API ■■■ urls_langchain.py # Routage URL
```

2.3 Composants Principaux

Composant	Fichier	Responsabilité	Technologies
Graph Manager	graph_manager.py	Orchestration centrale	LangGraph, TypedDict
Classifier Node	classifieur_node.py	Analyse des requêtes	Pydantic, Regex, LLM
Agent Nodes	agent_nodes.py	Traitement spécialisé	LangChain Tools, RAG
RAG Tools	rag_tools.py	Recherche vectorielle	ChromaDB, Embeddings
Django Bridge	django_integration.py	Intégration framework	Singleton, Configuration
Views	views_langchain.py	API endpoints	Django REST, JSON

3. Implémentation RAG

3.1 Système Multi-Domaines

Notre système RAG (Retrieval-Augmented Generation) implémente une architecture multi-domaines sophistiquée, permettant des recommandations spécialisées pour trois catégories distinctes de contenu littéraire.

Architecture RAG:

- **Séparation des domaines:** Collections ChromaDB dédiées
- **Embeddings spécialisés:** Modèles optimisés par type de contenu
- **Métadonnées riches:** Filtrage et ranking avancés
- **Recherche hybride:** Similarité sémantique + filtres
- **Fallback intelligent:** Recherche cross-domaine si nécessaire

Domaine	Collection ChromaDB	Source de Données	Métadonnées Clés
Tech Books	tech_books	TechBook (Django)	Langages, Difficulté, Notes
Literature	literature_books	LiteratureBook (Django)	Genres, Thèmes, Époque
Manga/BD	manga_comics_collection	CSV Files	Styles, Origine, Cible

3.2 ChromaDB et Embeddings

ChromaDB Configuration:

ChromaDB sert de base de données vectorielle pour stocker et rechercher les embeddings. Chaque domaine utilise une collection dédiée pour optimiser les performances et la pertinence.

Processus d'Embedding:

1. **Extraction:** Texte descriptif depuis les modèles Django
2. **Vectorisation:** Transformation en embeddings via modèle langage
3. **Indexation:** Stockage dans ChromaDB avec métadonnées
4. **Optimisation:** Index pour recherche rapide par similarité

Métadonnées Enrichies:

- **Tech:** programming_languages, difficulty_level, rating, price_range
- **Literature:** genres, themes, publication_year, target_audience
- **Manga:** style, origin_country, demographic, completed_status

Exemple d'implémentation ChromaDB:

```
# Exemple d'indexation ChromaDB
class TechRAGManager:
    def __init__(self):
        self.client = chromadb.PersistentClient(path="./chroma_db")
        self.collection = self.client.get_or_create_collection(
            name="tech_books",
            embedding_function=embedding_functions.DefaultEmbeddingFunction()
        )
    def add_books_batch(self, books):
        documents = []
        metadatas = []
        ids = []
        for book in books:
            # Construction du document texte
            doc_text = f"{book.title} {book.description} {book.summary}"
            documents.append(doc_text)
            # Métadonnées enrichies
            metadata = {
                "title": book.title,
                "programming_languages": book.programming_languages,
                "difficulty": book.difficulty_level,
                "rating": float(book.rating) if book.rating else 0.0
            }
            metadatas.append(metadata)
            ids.append(f"tech_book_{book.id}")
        # Indexation batch
        self.collection.add(
```

```
documents=documents, metadata=metadata, ids=ids )
```

3.3 Outils de Recherche

Les outils de recherche RAG implémentent l'interface LangChain BaseTool, permettant leur intégration seamless dans les workflows d'agents.

Tools Disponibles:

- **TechBookSearchTool:** Recherche de livres techniques
- **LiteratureBookSearchTool:** Recherche littéraire (exclut manga)
- **MangaContentSearchTool:** Recherche manga/BD unifiée
- **CombinedSearchTool:** Recherche multi-domaines

Caractéristiques Avancées:

- **Validation Pydantic:** Schémas stricts pour les entrées
- **Filtrage intelligent:** Exclusions et inclusions dynamiques
- **Scoring adaptatif:** Pondération par pertinence et métadonnées
- **Fallback gracieux:** Gestion des erreurs et alternatives
- **Limitation intelligente:** Contrôle du nombre de résultats

4. Analyse des Workflows

4.1 Classification Intelligente

Le système de classification hybride combine des règles déterministes avec l'intelligence artificielle pour une catégorisation précise et robuste des requêtes utilisateur.

Architecture de Classification:

1. **Analyse lexicale:** Détection de mots-clés avec seuils de confiance
2. **Expansion sémantique:** Enrichissement du vocabulaire détectable
3. **Classification LLM:** Analyse contextuelle pour cas ambigus
4. **Extraction de quantité:** Détection automatique du nombre souhaité
5. **Validation Pydantic:** Structure garantie de la sortie

Règles de Priorité:

- **Confiance ≥ 0.8 :** Classification directe sans LLM
- **Confiance < 0.8 :** Analyse LLM + validation
- **Ambiguïté:** Classification 'general' avec recherche multi-domaines
- **Erreur:** Fallback vers classification générale

Implémentation de la classification:

```
# Exemple de classification hybride
class ClassifieurNode:
    TECH_KEYWORDS = {
        'programming': 0.9, 'python': 0.95, 'javascript': 0.95, 'algorithm': 0.8,
        'data science': 0.9, 'machine learning': 0.95 }
    LITERATURE_KEYWORDS = {
        'roman': 0.9, 'poetry': 0.85, 'novel': 0.9, 'classic': 0.8, 'fiction': 0.7,
        'literature': 0.9 }
    MANGA_KEYWORDS = { 'manga': 0.95, 'anime': 0.9, 'bande dessinée': 0.95, 'comic': 0.8, 'bd': 0.95, 'shonen': 0.95 }

    def _classify_by_rules(self, query: str) -> Tuple[str, float]:
        query_lower = query.lower()
        max_confidence = 0.0
        best_category = "general"
        # Test de chaque catégorie
        for category, keywords in [ ("tech", self.TECH_KEYWORDS),
                                    ("literature", self.LITERATURE_KEYWORDS),
                                    ("manga", self.MANGA_KEYWORDS) ]:
            confidence = max( (score for keyword, score in keywords.items() if keyword in query_lower),
                              default=0.0 )
            if confidence > max_confidence:
                max_confidence = confidence
                best_category = category
        return best_category, max_confidence
```

4.2 Routage Adaptatif

Le RouterNode implémente un système de routage intelligent qui dirige chaque requête vers l'agent spécialisé optimal basé sur la classification et le contexte.

Logique de Routage:

- **Tech:** TechAgentNode pour programmation, algorithmique, outils
- **Literature:** LiteratureAgentNode pour romans, poésie, classiques
- **Manga:** MangaAgentNode pour manga, BD, comics, anime
- **General:** Recherche combinée avec fusion des résultats

Mécanismes Avancés:

- **Routage conditionnel:** Décision basée sur confiance et contexte
- **Fallback intelligent:** Redirection en cas d'échec d'agent
- **Load balancing:** Distribution équitable (future feature)
- **A/B testing:** Support pour expérimentations (planifié)

4.3 Formatage des Réponses

Le ResponseFormatterNode assure la cohérence et la qualité de toutes les réponses du système, quel que soit l'agent qui les a générées.

Responsabilités du Formatter:

- **Structuration:** Format JSON cohérent pour l'API
- **Validation:** Vérification de la complétude des réponses
- **Enrichissement:** Ajout de métadonnées (temps, agent, confiance)
- **Personnalisation:** Adaptation au style utilisateur
- **Fallback:** Messages d'erreur élégants

Structure de Réponse Standard:

- **recommendations:** Liste des livres recommandés
- **explanation:** Justification des choix
- **metadata:** Informations techniques (agent, temps, etc.)
- **suggestions:** Recommandations pour affiner la recherche
- **total_found:** Nombre total de résultats disponibles

5. Détails Techniques

5.1 Configuration

La configuration du système LangChain est centralisée dans les settings Django, permettant une gestion flexible des environnements et des paramètres.

```
# Configuration dans settings/development.py
LANGCHAIN_CONFIG = { #
    Configuration LLM 'provider': 'ollama', # openai, anthropic, ollama 'model':
    'llama3.2:3b', # Modèle spécifique 'ollama_base_url':
    'http://localhost:11434', # Paramètres de génération 'temperature': 0.7, #
    Créativité vs précision 'max_tokens': 512, # Limite de tokens de sortie
    'timeout': 30, # Timeout en secondes # Optimisations spécifiques
    'llama_optimized': True, # Optimisations Llama 'collect_stats': True, #
    Collecte de métriques 'debug_mode': False, # Mode debug détaillé #
    Configuration RAG 'rag_enabled': True, # Activation des RAG tools
    'max_rag_results': 5, # Limite résultats par RAG 'similarity_threshold': 0.7,
    # Seuil de similarité # Gestion d'erreurs 'fallback_enabled': True, # Fallback
    vers ancien système 'retry_attempts': 3, # Tentatives en cas d'échec
    'cache_enabled': True, # Cache des réponses (futur) } # Variables
    d'environnement pour secrets OPENAI_API_KEY = os.getenv('OPENAI_API_KEY')
    ANTHROPIC_API_KEY = os.getenv('ANTHROPIC_API_KEY')
```

5.2 Performance

Optimisations Implémentées:

- **Modèle local:** Llama 3.2 3B pour latence réduite
- **Batch processing:** Indexation RAG par lots de 100
- **Lazy loading:** Initialisation des composants à la demande
- **Connection pooling:** Réutilisation des connexions ChromaDB
- **Prompt optimization:** Templates optimisés pour tokens réduits

Métriques de Performance Typiques:

- **Classification:** ~200ms (règles) / ~800ms (LLM)
- **Recherche RAG:** ~150ms par domaine
- **Génération LLM:** ~2-5s selon la complexité
- **Formatage:** ~50ms
- **Total workflow:** ~3-8s bout-en-bout

Optimisations Futures:

- **Async/await:** Traitement asynchrone pour parallélisation
- **Response caching:** Cache intelligent des réponses fréquentes
- **Model quantization:** Réduction de taille pour vitesse accrue
- **Edge deployment:** Déploiement distribué pour latence globale

5.3 Monitoring

Système de Monitoring Intégré:

- **Health checks:** Endpoints de santé pour chaque composant
- **Performance tracking:** Mesure des temps de réponse
- **Error logging:** Journalisation détaillée des erreurs

- **Usage analytics:** Statistiques d'utilisation par agent
- **Quality metrics:** Évaluation de la pertinence des réponses

Endpoints de Monitoring:

- **/api/langchain/status/:** Statut global du système
- **/api/langchain/health/:** Health check détaillé
- **/api/langchain/metrics/:** Métriques de performance
- **/api/langchain/debug/:** Informations de debug (dev only)

6. Guide de Migration

6.1 Stratégie Progressive

La migration vers le système LangChain suit une approche progressive pour minimiser les risques et permettre un retour en arrière si nécessaire.

Phases de Migration:

1. **Phase 1:** Déploiement parallèle avec tests A/B
2. **Phase 2:** Migration graduelle par endpoint
3. **Phase 3:** Décommissioning de l'ancien système
4. **Phase 4:** Optimisations post-migration

Stratégie de Fallback:

- **Hybrid Views:** Test LangChain puis fallback automatique
- **Circuit breaker:** Désactivation automatique si erreurs
- **Manual override:** Contrôle admin pour forcer ancien système
- **Monitoring continu:** Alertes en cas de dégradation

Implémentation Vue Hybride:

```
# Exemple de Vue Hybride avec Fallback class HybridTechAgentChatView(APIView):
def post(self, request): try: # Tentative avec LangChain langchain_bridge =
DjangoLangChainBridge.get_instance() if langchain_bridge.is_available():
response = langchain_bridge.process_tech_query( request.data.get('query', ''))
) # Validation de la réponse if self._is_valid_response(response): return
Response({ 'source': 'langchain', 'response': response }) except Exception as
e: logger.warning(f"LangChain failed, fallback: {e}") # Fallback vers ancien
système old_manager = SimpleAgentManager() response =
old_manager.get_tech_recommendations( request.data.get('query', '')) return
Response({ 'source': 'fallback', 'response': response })
```

6.2 Compatibilité

Interfaces de Compatibilité:

- **LangChainAgentManager:** API compatible avec SimpleAgentManager
- **Response format:** Structure JSON identique à l'ancien système
- **Error handling:** Codes d'erreur et messages cohérents
- **Logging format:** Même format pour outils existants

Gestion des Différences:

- **Temps de réponse:** Adaptation des timeouts frontend
- **Nouvelles features:** Gestion gracieuse des champs additionnels
- **Métriques enrichies:** Backward compatibility pour analytics
- **Configuration:** Variables d'environnement non-breaking

6.3 Tests et Validation

Suite de Tests Complète:

- **Unit tests:** Tests unitaires pour chaque composant
- **Integration tests:** Tests d'intégration RAG ↔ LangChain

- **End-to-end tests:** Workflows complets utilisateur
- **Performance tests:** Benchmarks et tests de charge
- **Regression tests:** Non-régression vs ancien système

Validation de Qualité:

- **Response quality:** Évaluation humaine des recommandations
- **Relevance scoring:** Métriques automatiques de pertinence
- **User feedback:** Intégration des retours utilisateurs
- **A/B testing:** Comparaison statistique des performances

Métriques de Succès:

- **Précision:** >90% de recommandations pertinentes
- **Rapidité:** <8s pour 95% des requêtes
- **Disponibilité:** >99.5% uptime
- **Satisfaction:** Feedback utilisateur positif >85%

7. Annexes

7.1 Exemples de Code

```
# Exemple Complet d'Utilisation du Système from
agents.langchain_agents.django_integration import DjangoLangChainBridge # 1.
Initialisation du système bridge = DjangoLangChainBridge.get_instance() # 2.
Requête utilisateur user_query = "Je cherche 3 livres sur l'apprentissage du
Python pour débutants" # 3. Traitement via LangChain response =
bridge.process_query_with_routing(user_query) # 4. Structure de la réponse {
"recommendations": [ { "title": "Python Crash Course", "author": "Eric
Matthes", "description": "Introduction pratique à Python...", "rating": 4.8,
"difficulty": "beginner", "programming_languages": ["Python"],
"similarity_score": 0.95 } ], "explanation": "Ces livres ont été sélectionnés
pour leur approche...", "metadata": { "agent_used": "tech",
"processing_time": 3.2, "rag_results_count": 15, "classification_confidence":
0.92 }, "suggestions": [ "Vous pourriez aussi vous intéresser aux frameworks
web Python", "Considérez des projets pratiques pour consolider" ],
"total_found": 25 }
```

7.2 Configuration Complète

```
Configuration Complete - settings/production.py LANGCHAIN_CONFIG = {
Configuration LLM 'provider': 'ollama', 'model': 'llama3.2:3b', 'api_key':
os.getenv('LLM_API_KEY'), 'ollama_base_url': 'http://localhost:11434',
Parametres de Generation 'temperature': 0.7, 'max_tokens': 512, 'timeout': 30,
'top_p': 0.9, 'frequency_penalty': 0.1, Configuration RAG 'rag_enabled':
True, 'max_rag_results': 5, 'similarity_threshold': 0.7, 'rag_rerank': True,
Gestion d'Erreurs 'fallback_enabled': True, 'retry_attempts': 3,
'circuit_breaker_threshold': 5, 'circuit_breaker_timeout': 300, Performance
'llama_optimized': True, 'collect_stats': True, 'async_enabled': False,
'batch_size': 100, Debugging et Monitoring 'debug_mode': False, 'log_level':
'INFO', 'trace_requests': True, 'performance_logging': True, Cache (Futur)
'cache_enabled': False, 'cache_ttl': 3600, 'cache_backend': 'redis', Securite
'rate_limit': { 'enabled': True, 'requests_per_minute': 60, 'burst_limit': 10
}, 'input_validation': { 'max_query_length': 1000, 'sanitize_input': True,
'blocked_patterns': ['script', 'javascript'] } } Variables d'environnement
(secrets) OPENAI_API_KEY = os.getenv('OPENAI_API_KEY') ANTHROPIC_API_KEY =
os.getenv('ANTHROPIC_API_KEY') LANGCHAIN_DEBUG = os.getenv('LANGCHAIN_DEBUG',
'False').lower() == 'true'
```

7.3 Troubleshooting

Problèmes Courants et Solutions:

1. Ollama Connection Failed

- **Symptôme:** ConnectionError lors de l'appel LLM
- **Cause:** Ollama non démarré ou URL incorrecte
- **Solution:** Vérifier ollama serve et OLLAMA_BASE_URL

2. RAG Returns Empty Results

- **Symptôme:** Aucune recommandation trouvée
- **Cause:** ChromaDB non indexé ou seuil trop élevé
- **Solution:** Relancer indexation et ajuster similarity_threshold

3. Slow Response Times

- **Symptôme:** Réponses >10s
- **Cause:** Modèle trop gros ou RAG non optimisé
- **Solution:** Utiliser llama3.2:1b ou optimiser ChromaDB

4. Classification Errors

- **Symptôme:** Mauvais routage des requêtes
- **Cause:** Mots-clés manquants ou prompts inadéquats
- **Solution:** Enrichir KEYWORDS ou ajuster prompts

5. Memory Issues

- **Symptôme:** OOM lors de l'indexation
- **Cause:** Batch size trop élevé
- **Solution:** Réduire batch_size à 50 ou moins

Commandes de Debug Utiles:

- **Test connexion:** curl http://localhost:11434/api/version
- **Status système:** GET /api/langchain/status/
- **Logs détaillés:** LANGCHAIN_DEBUG=True dans .env
- **Reset ChromaDB:** rm -rf ./chroma_db && python manage.py index_all
- **Test classification:** python test_langchain_rag.py

Document généré automatiquement

Dual Book Advisor - Système LangChain

Pour plus d'informations, consulter la documentation technique interne.